

```

options {
    JAVA_UNICODE_ESCAPE = true;
    STATIC = false;
}

PARSER_BEGIN(MiniJavaParser)
    public class MiniJavaParser {}
PARSER_END(MiniJavaParser)

SKIP : /* WHITE SPACE */
{
    " "
|   "\t"
|   "\n"
|   "\r"
|   "\f"
}

SPECIAL_TOKEN : /* COMMENTS */
{
    <SINGLE_LINE_COMMENT: "//" (~["\n","\r"])* ("\"|\"r|\"r\n")>
|   <FORMAL_COMMENT: "/*" (~["*"])* "*" ("\" | (~["*","/"] (~["*"])* "*"))* "/">
|   <MULTI_LINE_COMMENT: "/*" (~["*"])* "*" ("\" | (~["*","/"] (~["*"])* "*"))* "/">
}

TOKEN :
{
    < LPAREN: "(" >
|   < RPAREN: ")" >
|   < LSQPAREN: "[" >
|   < RSQPAREN: "]" >
|   < LBRACE: "{" >
|   < RBRACE: "}" >
|   < SEMICOLON: ";" >
|   < DOT: "." >
|   < ASSIGN: "=" >
|   < LT: "<" >
|   < PLUS: "+" >
|   < MINUS: "-" >
|   < AND : "&&" >
|   < NOT : "!" >
|   < BOOLEAN: "boolean" >
|   < CLASS: "class" >
|   < INTERFACE: "interface" >
|   < ELSE: "else" >
|   < EXTENDS: "extends" >
|   < FALSE: "false" >
|   < IF: "if" >
|   < WHILE: "while" >
|   < INTEGER: "int" >
|   < LENGTH: "length" >
|   < MAIN: "main" >
|   < NEW: "new" >
|   < PUBLIC: "public" >
|   < RETURN: "return" >
|   < STATIC: "static" >
|   < STRING: "String" >
|   < THIS: "this" >
|   < TRUE: "true" >
|   < PRINT: "System.out.println" >
|   < VOID: "void" >
}

TOKEN : /* LITERALS */

```

```
{
  < INTEGER_LITERAL: ( [ "1"-"9" ] ( [ "0"-"9" ] ) * | "0" ) >
}
```

```
TOKEN : /* IDENTIFIERS */
```

```
{
  < IDENTIFIER: <LETTER> ( <LETTER> | <DIGIT> ) * >
  |
  < #LETTER:
    [
      "\u0024",
      "\u0041"-" \u005a",
      "\u005f",
      "\u0061"-" \u007a",
      "\u00c0"-" \u00d6",
      "\u00d8"-" \u00f6",
      "\u00f8"-" \u00ff",
      "\u0100"-" \u1fff",
      "\u3040"-" \u318f",
      "\u3300"-" \u337f",
      "\u3400"-" \u3d2d",
      "\u4e00"-" \u9fff",
      "\uf900"-" \ufaff"
    ]
  >
  |
  < #DIGIT:
    [
      "\u0030"-" \u0039",
      "\u0060"-" \u0069",
      "\u00f0"-" \u00f9",
      "\u0966"-" \u096f",
      "\u09e6"-" \u09ef",
      "\u0a66"-" \u0a6f",
      "\u0ae6"-" \u0aef",
      "\u0b66"-" \u0b6f",
      "\u0be7"-" \u0bef",
      "\u0c66"-" \u0c6f",
      "\u0ce6"-" \u0cef",
      "\u0d66"-" \u0d6f",
      "\u0e50"-" \u0e59",
      "\u0ed0"-" \u0ed9",
      "\u1040"-" \u1049"
    ]
  >
}
```

```
/*
 * The MiniJava Grammar Starts Here *
 */
```

```
void Goal() :
{
  MainClass()
  ( TypeDeclaration() ) *
  <EOF>
}
```

```
void MainClass() :
{
  "class" Identifier() "{"
```

```

    "public" "static" "void" "main" "(" "String" "[" "]" Identifier() ")"
    "{" ( LOOKAHEAD(2) VarDeclaration() )* ( Statement() )* "}"
    "}"
}

void TypeDeclaration() :
{}
{
    LOOKAHEAD(3)
    ClassDeclaration()
    |
    ClassExtendsDeclaration()
}

void ClassDeclaration() :
{}
{
    "class" Identifier() "{" ( VarDeclaration() )*
                        ( MethodDeclaration() )*
    "}"
}

void ClassExtendsDeclaration() :
{}
{
    "class" Identifier() "extends" Identifier()
                        "{" ( VarDeclaration() )*
                        ( MethodDeclaration() )*
    "}"
}

void VarDeclaration() :
{}
{
    Type() Identifier() ";"
}

void MethodDeclaration() :
{}
{
    "public" Type() Identifier()
    "(" ( FormalParameterList() )? ")"
    "{" ( LOOKAHEAD(2) VarDeclaration() )*
        ( Statement() )* "return" Expression() ";" "}"
}

void FormalParameterList() :
{}
{
    FormalParameter() FormalParameterTail()
}

void FormalParameter() :
{}
{
    Type() Identifier()
}

void FormalParameterTail() :
{}
{
    ( FormalParameterTerm() )*
}

void FormalParameterTerm() :
```

```
{  
{  
  "," FormalParameter()  
}  
  
void Type() :  
{  
{  
  LOOKAHEAD(3)  
  ArrayType()  
|  
  BooleanType()  
|  
  IntegerType()  
|  
  Identifier()  
}  
}  
  
void ArrayType() :  
{  
{  
  BooleanArrayType()  
|  
  IntegerArrayType()  
}  
}  
  
void BooleanArrayType() :  
{  
{  
  "boolean" "[" "]"  
}  
}  
  
void IntegerArrayType() :  
{  
{  
  "int" "[" "]"  
}  
}  
  
void BooleanType() :  
{  
{  
  "boolean"  
}  
}  
  
void IntegerType() :  
{  
{  
  "int"  
}  
}  
  
void Statement() :  
{  
{  
  Block()  
|  
  LOOKAHEAD(2)  
  AssignmentStatement()  
|  
  LOOKAHEAD(2)  
  ArrayAssignmentStatement()  
|  
  IfStatement()  
|  
  WhileStatement()  
|  
}
```

```
PrintStatement()
}

void Block() :
{}
{
    "{" ( Statement() ) * "}"
}

void AssignmentStatement() :
{}
{
    Identifier() "=" Expression() ";"
}

void ArrayAssignmentStatement() :
{}
{
    Identifier() "[" Expression() "]" "=" Expression() ";"
}

void IfStatement() :
{}
{
    "if" "(" Expression() ")" Statement() "else" Statement()
}

void WhileStatement() :
{}
{
    "while" "(" Expression() ")" Statement()
}

void PrintStatement() :
{}
{
    "System.out.println" "(" Expression() ")" ";"
}

void Expression() :
{}
{
    LOOKAHEAD( Clause() "&&" )
    AndExpression()
|
    LOOKAHEAD( PrimaryExpression() "<" )
    CompareExpression()
|
    LOOKAHEAD( PrimaryExpression() "+" )
    PlusExpression()
|
    LOOKAHEAD( PrimaryExpression() "-" )
    MinusExpression()
|
    LOOKAHEAD( PrimaryExpression() "*" )
    TimesExpression()
|
    LOOKAHEAD( PrimaryExpression() "[" )
    ArrayLookup()
|
    LOOKAHEAD( PrimaryExpression() "." "length" )
    ArrayLength()
|
    LOOKAHEAD( PrimaryExpression() "." Identifier() "(" )
    MessageSend()
```

```
|
  Clause()
}

void AndExpression() :
{}
{
  Clause() "&&" Clause()
}

void CompareExpression() :
{}
{
  PrimaryExpression() "<" PrimaryExpression()
}

void PlusExpression() :
{}
{
  PrimaryExpression() "+" PrimaryExpression()
}

void MinusExpression() :
{}
{
  PrimaryExpression() "-" PrimaryExpression()
}

void TimesExpression() :
{}
{
  PrimaryExpression() "*" PrimaryExpression()
}

void ArrayLookup() :
{}
{
  PrimaryExpression() "[" PrimaryExpression() "]"
}

void ArrayLength() :
{}
{
  PrimaryExpression() "." "length"
}

void MessageSend() :
{}
{
  PrimaryExpression() "." Identifier()
  "(" ( ExpressionList() )? ")"
}

void ExpressionList() :
{}
{
  Expression() ExpressionTail()
}

void ExpressionTail() :
{}
{
  ( ExpressionTerm() ) *
}
```

```
void ExpressionTerm() :
{
    "," Expression()
}

void Clause() :
{
    {
        NotExpression()
    | PrimaryExpression()
    }
}

void PrimaryExpression() :
{
    {
        IntegerLiteral()
    | TrueLiteral()
    | FalseLiteral()
    | Identifier()
    | ThisExpression()
    | LOOKAHEAD(3)
      ArrayAllocationExpression()
    | AllocationExpression()
    | BracketExpression()
    }
}

void IntegerLiteral() :
{
    {
        <INTEGER_LITERAL>
    }
}

void TrueLiteral() :
{
    {
        "true"
    }
}

void FalseLiteral() :
{
    {
        "false"
    }
}

void Identifier() :
{
    {
        <IDENTIFIER>
    }
}

void ThisExpression() :
{
    {
        "this"
    }
}
```

```
void ArrayAllocationExpression() :
{
{
    LOOKAHEAD(3)
    BooleanArrayAllocationExpression()
|
    LOOKAHEAD(3)
    IntegerArrayAllocationExpression()
}

void BooleanArrayAllocationExpression() :
{
{
    "new" "boolean" "[" Expression() "]"
}

void IntegerArrayAllocationExpression() :
{
{
    "new" "int" "[" Expression() "]"
}

void AllocationExpression() :
{
{
    "new" Identifier() "(" ")"
}

void NotExpression() :
{
{
    "!" Clause()
}

void BracketExpression() :
{
{
    "(" Expression() ")"
}
```